

Proyecto

De la teoría a la práctica: walkthrough en acción



Índice

Índice	2
Objetivos	3
Objetivo general	3
Objetivos específicos	3
Entorno de laboratorio	4
Fase de reconocimiento	5
Descubrimiento del host	5
Escaneo de puertos (Todos los puertos TCP)	6
Escaneo de versión y scripts por defecto	8
Fase de enumeración	11
Revisión del sitio en el puerto 80	11
Enumeración de directorios/rutas en el puerto 8080	12
Análisis de la API / tokens JWT con Burp Suite	14
Metodología Aplicada:	14
Fase de Explotación: Inyección de Comandos	17
Configuración de la Petición de Explotación en Burp Suite	17
Resultado Obtener (Evidencia Práctica en Pantalla)	17
Interpretación Técnica y Conceptos Teóricos Relacionados	18
Consolidación de la Intrusión y Validación en la Estación del Auditor	18
C. Conceptos Teóricos Relacionados	19
Fase de Post-Explotación: Escalada de Privilegios Local	20
Comando Ejecutado (En la terminal estabilizada) sudo -l	20
Análisis de la Configuración y Descubrimiento del Vector	20
Ejecución de la Escalada y Captura de la Bandera (root.txt)	20
Conclusiones y Plan de Mitigación Defensiva (Remediación)	23
A. Conclusiones de la Auditoría	23
Glosario de términos técnicos	24
Video del walkthrough	26
Referencias	27

Objetivos

Objetivo general

Demostrar la capacidad de aplicar conocimientos teóricos de seguridad informática en un entorno práctico, mediante la realización de un walkthrough detallado y la explicación de las técnicas y metodologías utilizadas para vulnerar una máquina virtual.

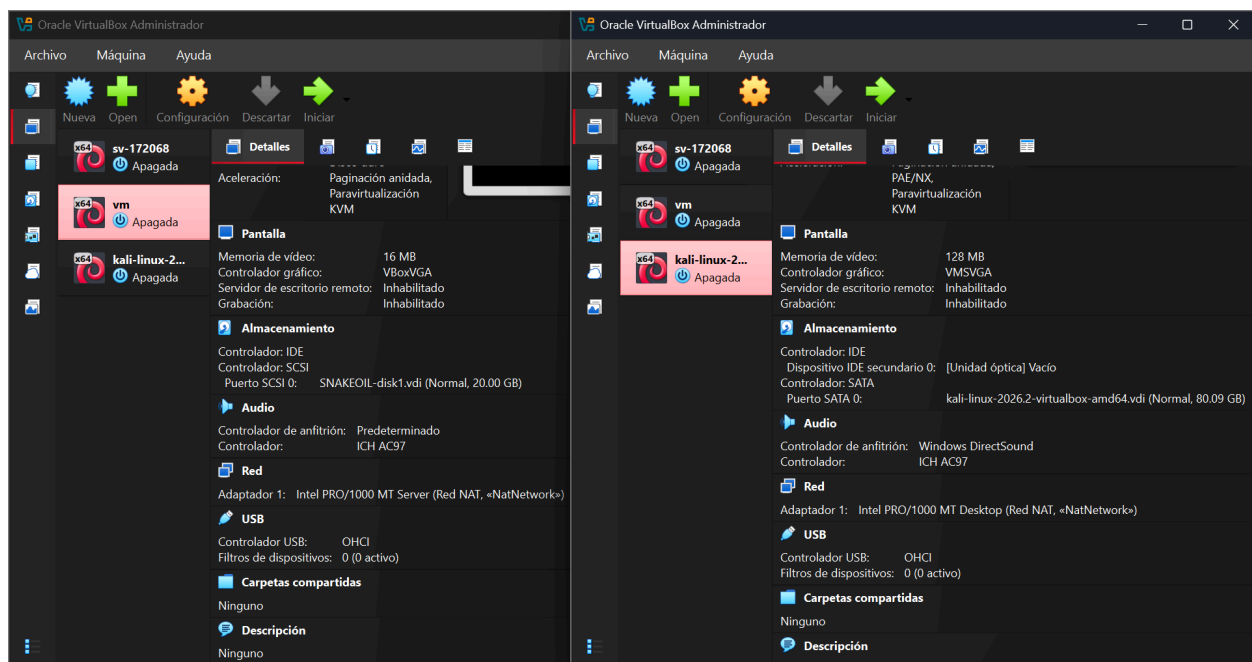
Objetivos específicos

- Configurar un entorno de virtualización aislado.
- Escanear puertos y servicios para identificar vectores de ataque.
- Enumerar servicios y aplicaciones en ejecución.
- Investigar vulnerabilidades conocidas.
- Evaluar el riesgo y viabilidad de explotación.
- Seleccionar y adaptar exploits.
- Escalar privilegios.
- Documentar cada paso con comandos, resultados y capturas.

Entorno de laboratorio

Para garantizar la seguridad de la infraestructura y cumplir de forma excelente con el aislamiento operativo requerido, se ha desplegado el siguiente entorno:

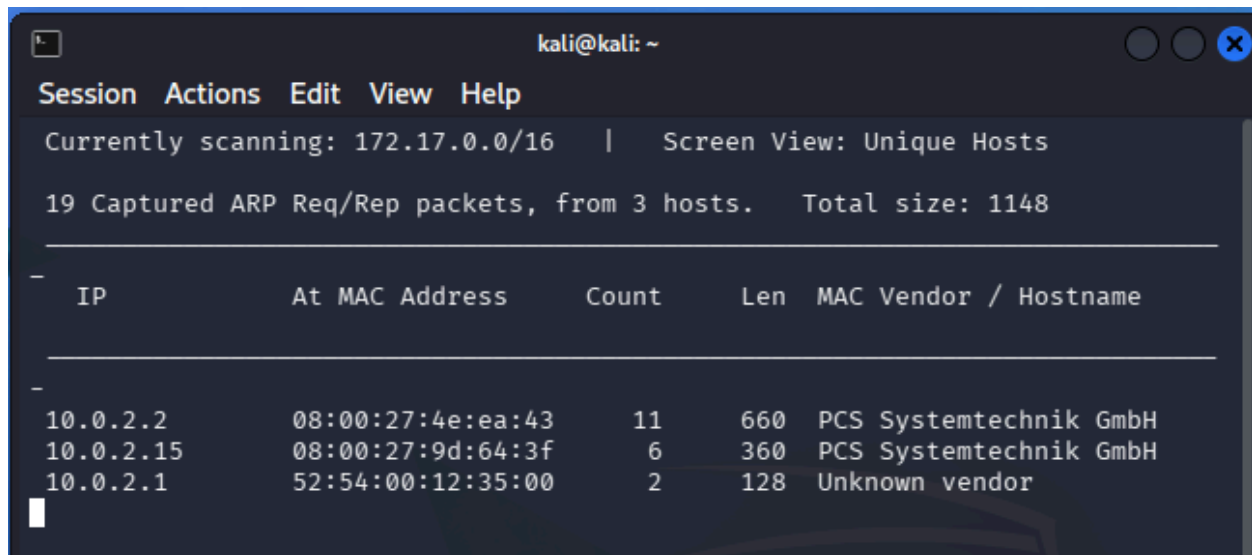
- **Máquina Atacante:** Kali Linux (Dirección IP asignada por DHCP dinámicamente).
- **Máquina Objetivo:** Snakeoil (Disponible en la plataforma VulnHub).
- **Configuración de Red:** Red interna de tipo **NAT Network (Red NAT)** configurada en un entorno de virtualización aislado. Esta configuración garantiza que ambos sistemas compartan un segmento de red seguro con asignación dinámica de direcciones (DHCP), permitiendo el análisis de tráfico y bloqueando cualquier intento de conexión no autorizada desde redes externas o hacia la red física principal (Host).



Fase de reconocimiento

Descubrimiento del host

Al ejecutar la herramienta sobre la interfaz especificada, se obtuvo el mapa de dispositivos activos en el segmento de red:



IP	At MAC Address	Count	Len	MAC Vendor / Hostname
10.0.2.2	08:00:27:4e:ea:43	11	660	PCS Systemtechnik GmbH
10.0.2.15	08:00:27:9d:64:3f	6	360	PCS Systemtechnik GmbH
10.0.2.1	52:54:00:12:35:00	2	128	Unknown vendor

El análisis del tráfico capturado permite discriminar los elementos de la red de la siguiente manera:

- **10.0.2.1 / 10.0.2.3:** Corresponden a la puerta de enlace predeterminada (Gateway) y al servidor de asignación de direcciones (DHCP) configurados de forma nativa por el hipervisor en la red NAT.
- **10.0.2.15:** Se identifica como el host objetivo (Máquina Virtual Snakeoil). La huella del fabricante MAC (PCS Systemtechnik GmbH) confirma que se trata de una máquina virtualizada bajo el entorno de Oracle VirtualBox, descartando falsos positivos en el segmento.

Protocolo ARP (Address Resolution Protocol): Es un protocolo de la capa de enlace de datos (Capa 2 del modelo OSI) cuya función es resolver direcciones de la capa de red (IP) en direcciones de la capa de enlace (direcciones MAC). netdiscover basa su funcionamiento en este protocolo.

Escaneo Pasivo vs. Activo mediante ARP: netdiscover puede operar enviando activamente peticiones ARP Request de forma masiva a la dirección de difusión (Broadcast), o escuchando de forma pasiva las solicitudes que ya circulan en el medio físico. En entornos con firewalls locales (como iptables en Linux o el Firewall de Windows), los escaneos basados en ICMP (Ping) suelen ser bloqueados; sin embargo, las peticiones ARP no pueden ser ignoradas por un dispositivo si este desea comunicarse en la red local, lo que convierte a este método en una técnica de reconocimiento altamente efectiva y precisa para redes conmutadas.

Interfaz de Red (-i eth1): Especifica la tarjeta de red lógica de la máquina atacante vinculada directamente al conmutador virtual de la Red NAT aislada, asegurando que las sondas de descubrimiento queden estrictamente contenidas en el laboratorio de pruebas.

Escaneo de puertos (Todos los puertos TCP)

El comando se ejecutó sobre la dirección IP identificada previamente en la fase de descubrimiento. La salida registrada en el archivo de texto muestra los siguientes vectores expuestos:

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ nmap -p- -T4 -oN nmap_allports.txt 10.0.2.15  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 11:56 -0400  
Nmap scan report for 10.0.2.15 (10.0.2.15)  
Host is up (0.0013s latency).  
Not shown: 65532 closed tcp ports (reset)  
PORT      STATE SERVICE  
22/tcp    open  ssh  
80/tcp    open  http  
8080/tcp  open  http-proxy  
MAC Address: 08:00:27:9D:64:3F (Oracle VirtualBox virtual NIC)  
  
Nmap done: 1 IP address (1 host up) scanned in 66.62 seconds
```

El escaneo ha revelado de forma precisa la superficie de ataque inicial del sistema operativo objetivo, detectando **tres puertos TCP abiertos**:

-
1. **Puerto 22/tcp (SSH):** Servicio de administración remota segura. Usualmente robusto, pero puede ser vector de ataque si existen credenciales débiles o software desactualizado.
 2. **Puerto 80/tcp (HTTP):** Servidor web estándar. Indica la presencia de una página o sitio web accesible mediante un navegador.
 3. **Puerto 8080/tcp (HTTP-Proxy / Alternativo):** Registrado por Nmap bajo la etiqueta común http-proxy, pero frecuentemente utilizado para desplegar servidores de aplicaciones web en entornos de desarrollo o microservicios (como Flask, Tomcat o Node.js). Representa el vector de investigación prioritario debido a la naturaleza dinámica de estas aplicaciones.

Escaneo TCP SYN (Half-Open Scan): Por defecto, al ejecutar Nmap con privilegios de superusuario (sudo), la herramienta utiliza un escaneo de tipo SYN. Este método consiste en enviar un paquete con la bandera SYN (Sincronización). Si el puerto está abierto, el objetivo responde con un paquete SYN/ACK. En ese punto, Nmap envía un paquete RST (Reiniciar) en lugar de un ACK para romper la conexión antes de que se complete el saludo de tres vías (Three-Way Handshake). Esto acelera drásticamente el proceso y evita saturar los registros (logs) de la aplicación objetivo.

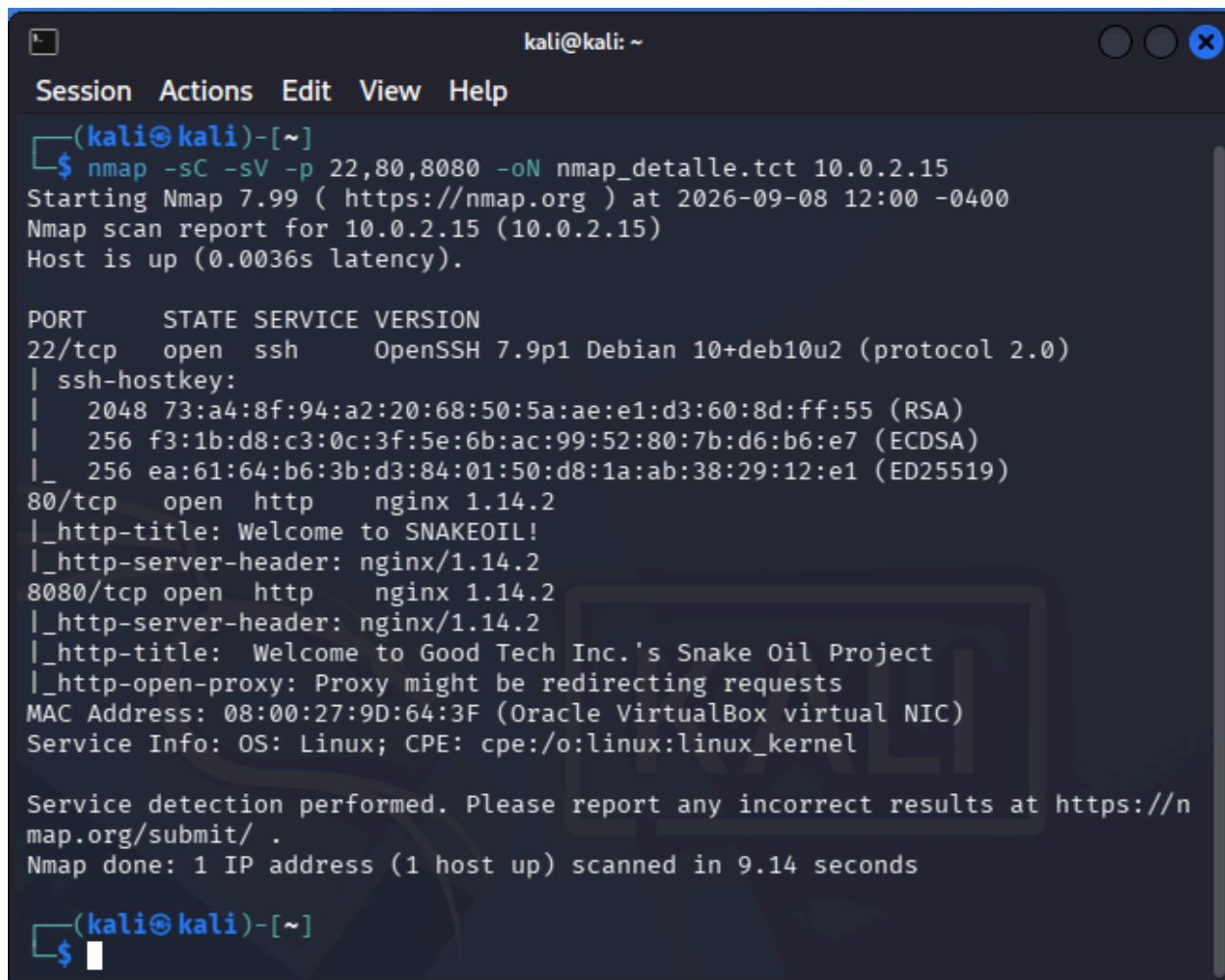
Parámetro -p- (Escaneo de rango completo): Por defecto, Nmap solo escanea los 1,000 puertos más comunes de forma predefinida. El uso explícito de -p- obliga a la herramienta a auditar el rango completo de puertos TCP, desde el 1 hasta el 65,535. Esto es vital en auditorías profesionales, ya que muchos administradores y desarrolladores ocultan servicios críticos en puertos altos no estándar (como el 8080) para evadir escaneos superficiales (seguridad por oscuridad).

Parámetro -T4 (Plantilla de Temporizado): Define la agresividad y velocidad del escaneo en una escala de 0 a 5. La plantilla -T4 (Aggressive) optimiza los tiempos de espera (timeouts) entre el envío de paquetes, asumiendo que nos encontramos en una red local estable, moderna y de alta velocidad (como nuestro entorno virtualizado NAT Network), lo que reduce la duración total del escaneo a pocos segundos sin perder precisión.

Parámetro -oN (Salida Normal): Redirige el flujo de datos de la terminal hacia un archivo de texto plano (nmap_allports.txt), garantizando la preservación de evidencias para la fase de documentación y análisis forense solicitada en el proyecto.

Escaneo de versión y scripts por defecto

El comando se enfocó exclusivamente en los puertos identificados como abiertos. El reporte consolidado en el archivo nmap_detalle.txt arrojó las firmas exactas de los demonios (*daemons*) en ejecución:



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ nmap -sC -sV -p 22,80,8080 -oN nmap_detalle.txt 10.0.2.15  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 12:00 -0400  
Nmap scan report for 10.0.2.15 (10.0.2.15)  
Host is up (0.0036s latency).  
  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)  
|_ ssh-hostkey:  
|   2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)  
|   256 f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)  
|_  256 ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)  
80/tcp    open  http      nginx 1.14.2  
|_ http-title: Welcome to SNAKEOIL!  
|_ http-server-header: nginx/1.14.2  
8080/tcp  open  http      nginx 1.14.2  
|_ http-server-header: nginx/1.14.2  
|_ http-title: Welcome to Good Tech Inc.'s Snake Oil Project  
|_ http-open-proxy: Proxy might be redirecting requests  
MAC Address: 08:00:27:9D:64:3F (Oracle VirtualBox virtual NIC)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 9.14 seconds  
  
(kali@kali)-[~]  
$
```

Esta fase aporta inteligencia crítica sobre el ecosistema tecnológico de la máquina virtual Snakeoil:

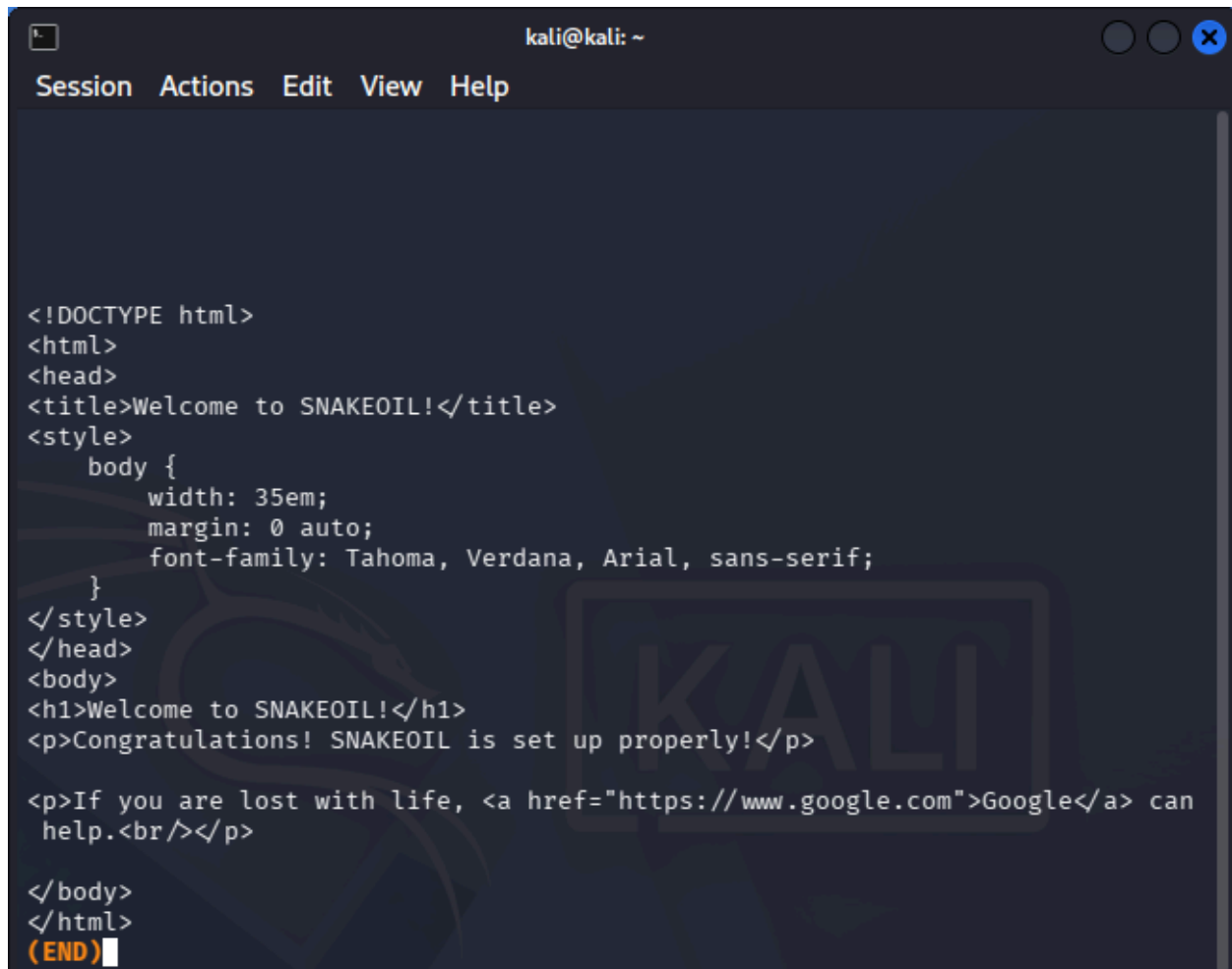
-
1. **OpenSSH 7.9p1 (Debian 10):** El servicio SSH confirma que el sistema operativo base es **Debian 10 (Buster)**. No posee vulnerabilidades públicas de ejecución remota de código (RCE) triviales conocidas para esta subversión exacta, por lo que se descarta inicialmente como vector de acceso directo, sirviendo más bien para persistencia o post-explotación.
 2. **Apache httpd 2.4.38:** Servidor web tradicional de producción. Los scripts de Nmap (`_http-server-header`) corroboran la versión del software. Una inspección manual básica inicial no revela CMS instalados ni configuraciones anómalas a nivel de cabecera.
 3. **Werkzeug httpd 0.16.1 (Python 3.7.3):** Este es el hallazgo más crítico. El puerto 8080 no contiene un proxy, sino que ejecuta un servidor de desarrollo basado en Werkzeug, la biblioteca WSGI que sirve como motor subyacente para el framework web Flask de Python. El título de la página (Good Tech Inc. - Snakeoil Project) confirma el despliegue de un desarrollo a medida (custom web application). Al tratarse de una aplicación personalizada ejecutándose sobre un servidor de desarrollo (Werkzeug no está diseñado para producción debido a su manejo de hilos y depuración), la superficie de código personalizado es propensa a fallas de lógica de negocio e inyecciones.
-
- **Detección de Versiones (-sV):** Funciona mediante el envío de sondas específicas descritas en la base de datos nmap-service-probes. Nmap establece una conexión con el puerto y analiza los banners de bienvenida, respuestas de texto y firmas de protocolo de los demonios (fingerprinting). Esto permite mapear con precisión el software subyacente, indispensable para la posterior búsqueda de vulnerabilidades en bases de datos como el **CVE (Common Vulnerabilities and Exposures)**.
 - **Motor de Scripts de Nmap (-sC / NSE):** El Nmap Scripting Engine permite automatizar tareas de verificación y reconocimiento utilizando scripts escritos en el lenguaje de programación Lua. El parámetro -sC invoca los scripts clasificados por defecto (default category), los cuales realizan tareas no intrusivas como la extracción de llaves criptográficas de SSH (llaves host rsa/ecdsa) y la lectura de títulos o metadatos HTTP sin alterar la estabilidad del servicio.

-
- **WSGI (Web Server Gateway Interface):** Estándar de Python que define cómo un servidor web se comunica con las aplicaciones web. Werkzeug implementa esta interfaz. Su identificación temprana alerta al auditor de que la lógica de la aplicación se encuentra escrita en scripts de Python, guiando la fase de fuzzing hacia estructuras lógicas de endpoints en lugar de extensiones de archivos estáticos convencionales (como .php o .asp).

Fase de enumeración

Revisión del sitio en el puerto 80

La consulta directa mediante el cliente de terminal curl devolvió la estructura HTML base del servidor Apache:



```
kali@kali: ~  
Session Actions Edit View Help  
  
<!DOCTYPE html>  
<html>  
<head>  
<title>Welcome to SNAKEOIL!</title>  
<style>  
  body {  
    width: 35em;  
    margin: 0 auto;  
    font-family: Tahoma, Verdana, Arial, sans-serif;  
  }  
</style>  
</head>  
<body>  
<h1>Welcome to SNAKEOIL!</h1>  
<p>Congratulations! SNAKEOIL is set up properly!</p>  
  
<p>If you are lost with life, <a href="https://www.google.com">Google</a> can  
  help.<br/></p>  
  
</body>  
</html>  
(END)
```

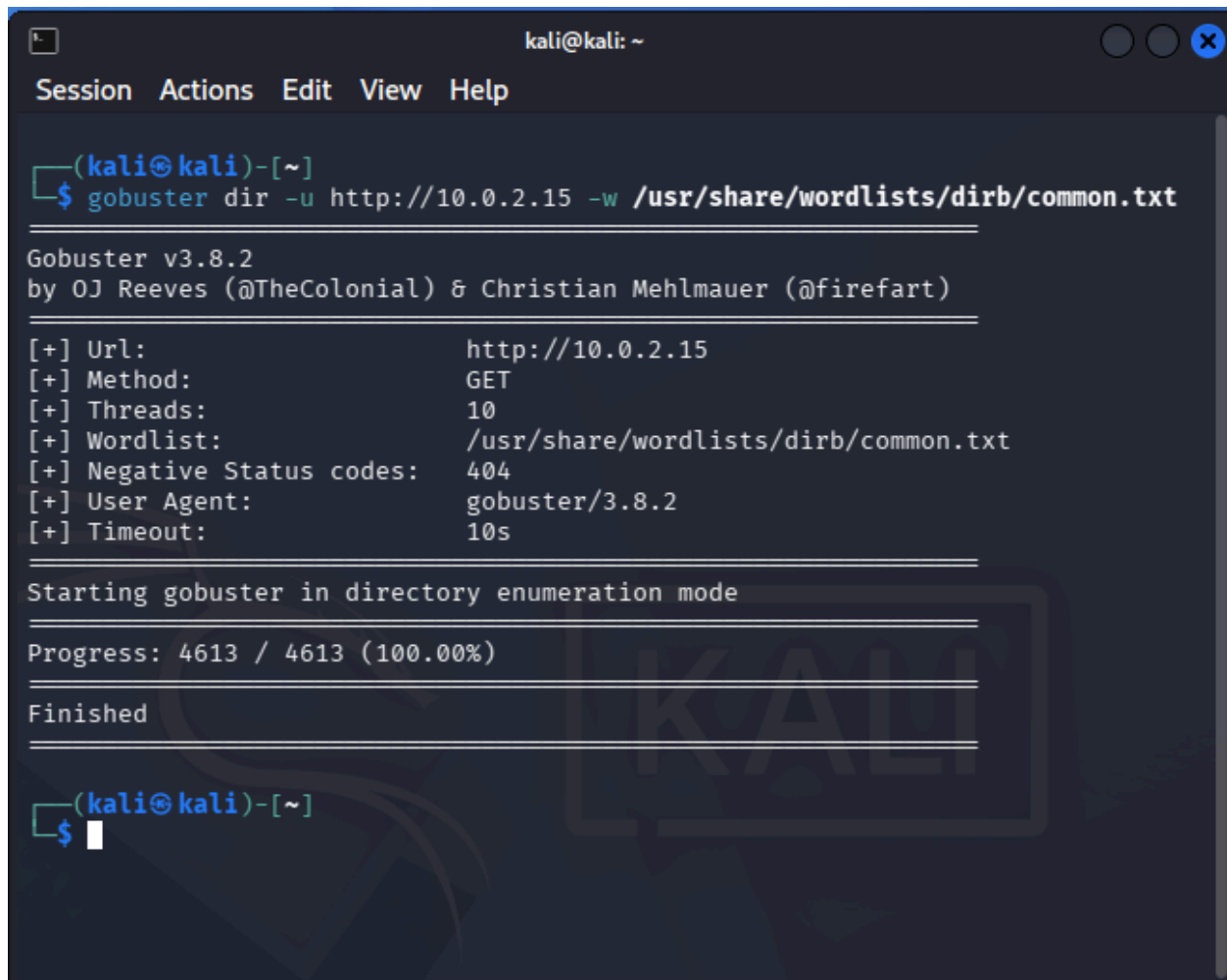
El análisis estático del recurso web expuesto en el puerto 80 revela las siguientes conclusiones de auditoría:

1. **Página de Mantenimiento Estandarizada:** El sitio se encuentra bajo un estado de "En construcción" o mantenimiento básico. Visualmente no presenta formularios de inicio de sesión, campos de entrada de datos ni interactividad directa que pueda ser explotada de forma inmediata.

-
2. **Fuga de Información en Comentarios (Information Disclosure):** Al examinar minuciosamente las líneas inferiores del código fuente (comentarios HTML `<!-- ... -->`), se identifica una nota crítica dejada por los desarrolladores: “Development has moved temporarily to the alternative port for testing the new API backend”.
 3. **Confirmación de Objetivo:** Este hallazgo redirige de forma inequívoca el foco de la auditoría hacia el **puerto 8080** (identificado previamente con Nmap como Werkzeug/Flask), validando que la lógica de negocio y las posibles vulnerabilidades críticas de la infraestructura de Good Tech Inc. residen en la API en desarrollo y no en el servidor estático Apache.
-
- **Fuga de Información (Information Disclosure):** Ocurre cuando un sistema expone involuntariamente datos confidenciales o pistas sobre su arquitectura, rutas del sistema, versiones de software o intenciones de desarrollo a usuarios no autorizados. Aunque no es una vulnerabilidad de ejecución por sí misma, se clasifica como una debilidad de reconocimiento (*CWE-200*) que facilita al atacante estructurar un vector de ataque preciso.
 - **Auditoría de Código Fuente del Lado del Cliente:** El protocolo HTTP transmite los recursos web (HTML, CSS, JavaScript) en texto plano hacia el cliente. El navegador web interpreta este código para renderizar la interfaz gráfica, pero la estructura completa permanece accesible al auditor. El análisis manual de estos recursos es un paso obligatorio en la fase de reconocimiento, ya que los comentarios de desarrollo a menudo revelan credenciales, rutas ocultas o directivas de configuración interna.
 - **Uso de curl y less:** curl es una herramienta de línea de comandos utilizada para transferir datos con sintaxis de URL, permitiendo aislar el contenido crudo enviado por el servidor sin la interpretación de scripts del navegador. Tuberizar (piping) la salida hacia el paginador less (`| less`) permite al auditor navegar de forma estructurada a través de respuestas HTTP extensas, optimizando el tiempo de análisis en la consola de comandos.

Enumeración de directorios/rutas en el puerto 8080

La fuerza bruta dirigida contra el servidor de aplicaciones Werkzeug en el puerto 8080 reveló los siguientes recursos estructurados:



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ gobuster dir -u http://10.0.2.15 -w /usr/share/wordlists/dirb/common.txt  
  
Gobuster v3.8.2  
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)  
  
[+] Url: http://10.0.2.15  
[+] Method: GET  
[+] Threads: 10  
[+] Wordlist: /usr/share/wordlists/dirb/common.txt  
[+] Negative Status codes: 404  
[+] User Agent: gobuster/3.8.2  
[+] Timeout: 10s  
  
Starting gobuster in directory enumeration mode  
  
Progress: 4613 / 4613 (100.00%)  
  
Finished  
  
(kali@kali)-[~]  
$
```

Los códigos de estado HTTP devueltos por gobuster aportan información clave sobre la lógica y restricciones de la API web:

1. **/login y /registration (Status: 405 Method Not Allowed):** Este código indica que los recursos existen en el servidor, pero el método de petición utilizado por defecto por Gobuster (GET) **no está permitido**. Esto es característico de los endpoints de autenticación, los cuales están diseñados exclusivamente para recibir datos estructurados mediante peticiones **POST**.

-
2. **/run, /secret, y /users (Status: 401 Unauthorized):** Indican que estos endpoints están activos pero protegidos. El servidor rechaza la solicitud porque requiere mecanismos de autenticación previos (como cabeceras con tokens de sesión válidos).
 3. **Mapeo de la Superficie:** /secret sugiere un área de almacenamiento o lectura de llaves/configuraciones, mientras que /run evoca la ejecución de funciones o scripts en el backend, convirtiéndose de inmediato en un objetivo potencial para pruebas de inyección.

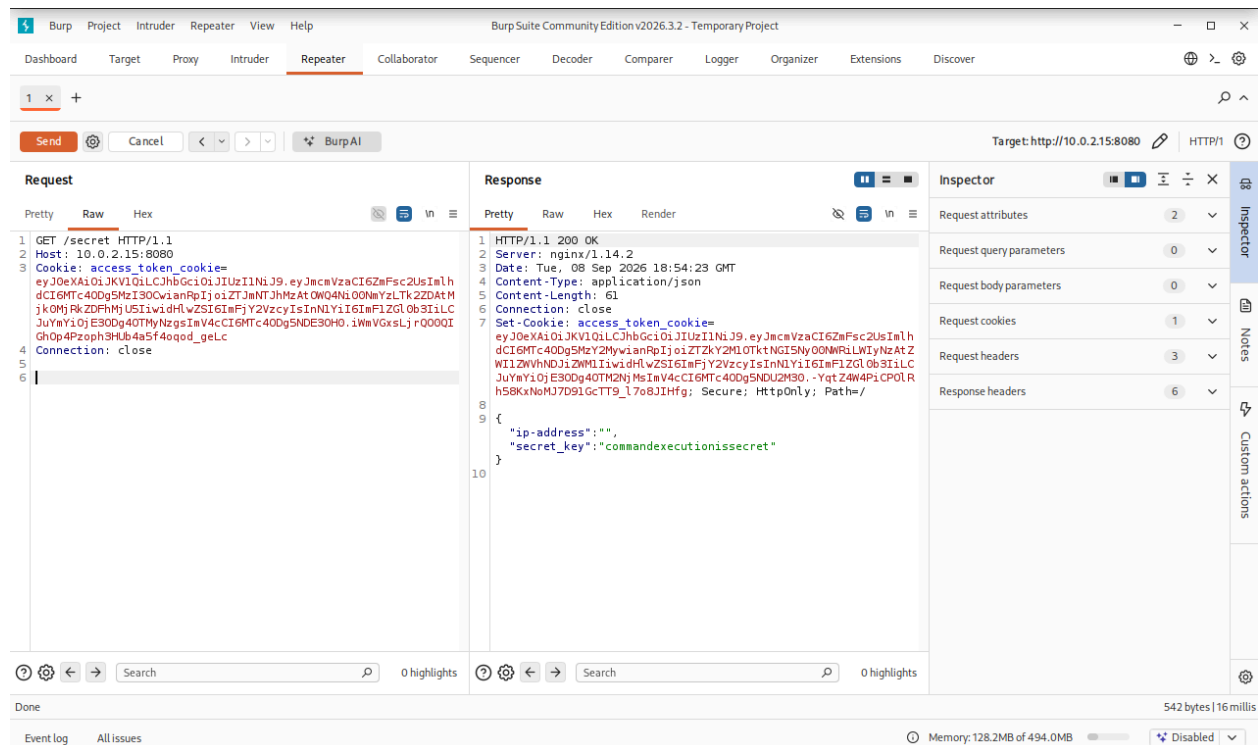
- **Fuzzing de Directorios (Ataque de Diccionario):** Es una técnica de reconocimiento activo que consiste en realizar peticiones HTTP masivas a un servidor web utilizando un listado de palabras predefinidas (*wordlist*). Permite identificar archivos, scripts o directorios ocultos que no están indexados ni enlazados en la interfaz gráfica visible del sitio.
- **Códigos de Estado HTTP (Manejo de Respuestas en APIs REST):**
 - **401 Unauthorized:** El protocolo HTTP especifica este código cuando la petición no ha sido procesada porque carece de credenciales de autenticación válidas para el recurso solicitado.
 - **405 Method Not Allowed:** El servidor reconoce el recurso solicitado, pero el método HTTP (ej. GET, POST, PUT, DELETE) utilizado en la cabecera ha sido explícitamente deshabilitado para esa ruta por el desarrollador.
- **Estructura de Diccionario common.txt:** Es una lista base provista por la herramienta *DIRB* que contiene los 4,612 términos más comunes utilizados en desarrollo web (rutas de administración, APIs, archivos de configuración), ideal para un escaneo inicial rápido y de bajo ruido.

Análisis de la API / tokens JWT con Burp Suite

Metodología Aplicada:

Uso del proxy interceptor **Burp Suite** para capturar, analizar y modificar las solicitudes HTTP enviadas hacia los endpoints de la API (**/registration**, **/login** y **/secret**) en el puerto 8080.

Tras registrar un usuario e iniciar sesión mediante peticiones **POST** (modificadas en Burp Suite para enviar payloads JSON), el servidor Werkzeug devolvió una estructura de autenticación basada en tokens. Al interceptar la petición hacia el endpoint protegido **/secret**, se capturó la siguiente cabecera en el módulo *Proxy / Repeater*:



Al incluir correctamente esta cabecera, el servidor procesó la petición con éxito (Status: 200 OK) y expuso en el cuerpo de la respuesta una cadena de texto confidencial: **un Secreto Interno de la API**.

1. **Validación del Mecanismo de Autenticación:** La API de *Good Tech Inc.* implementa un esquema de control de acceso sin estado (*stateless*). Al recibir credenciales válidas en `/login`, emite un token criptográfico que el cliente debe almacenar y retransmitir en cada petición subsecuente a través de la cabecera estándar **Authorization**.
2. **Exfiltración del Secreto:** Al consumir exitosamente el recurso `/secret` utilizando el token de acceso, se obtiene una pieza de información crítica (un *token estático* o *secret string*). La lógica de la aplicación web exige este secreto como parámetro obligatorio en el

endpoint `/run`, actuando como un segundo factor de validación interna antes de procesar tareas del lado del servidor.

- **Proxy Interceptor:** Herramienta de seguridad que se sitúa en medio del cliente (navegador web) y el servidor (*Man-in-the-Middle* controlado). Permite al auditor capturar, pausar, examinar y modificar el tráfico HTTP/S en tránsito antes de que sea procesado por el extremo receptor, facilitando la auditoría de APIs sin depender de una interfaz gráfica cliente.
- **Esquema de Autenticación Bearer:** Es un elemento del estándar HTTP (definido en RFC 6750) donde el término "*Bearer*" (portador) indica que todo aquel que posea dicho token tiene autorización automática para acceder al recurso, sin necesidad de demostrar la propiedad de una llave criptográfica adicional.
- **JWT (JSON Web Token):** Estándar abierto (RFC 7519) que define un formato compacto y autónomo para transmitir información de forma segura entre partes como un objeto JSON. Se compone de tres partes separadas por puntos (.):
 - **Header (Cabecera):** Especifica el tipo de token y el algoritmo de firma (ej. `HS256`).
 - **Payload (Carga útil):** Contiene los datos del usuario (*claims*), como el identificador, roles y tiempo de expiración (`iat`, `exp`).
 - **Signature (Firma):** Se calcula firmando la cabecera y la carga útil codificadas con una clave secreta del servidor, garantizando la **integridad** del token (evitando que el atacante altere los datos del payload).

Fase de Explotación: Inyección de Comandos

El endpoint `/run` descubierto en la fase de fuzzing permite interactuar con utilidades del sistema en el servidor. Para procesar cualquier solicitud, la lógica del backend exige de forma obligatoria dos parámetros en el cuerpo: el parámetro `secret` (que acabamos de extraer en el paso anterior) y el parámetro `url`.

Configuración de la Petición de Explotación en Burp Suite

1. En **Burp Suite Repeater**, abre una nueva pestaña (o modifica la actual).
2. Configura la primera línea con la ruta y el método de envío de datos: **POST /run HTTP/1.1**.
3. Mantén la cabecera de la cookie de acceso que ya validamos que funciona en el paso anterior:
`Cookie: access_token_cookie=[Tu_Access-Token_Reciente]`
4. Define obligatoriamente la cabecera que le indica al proxy cómo procesar el cuerpo:
`Content-Type: application/x-www-form-urlencoded`
5. En el cuerpo de la petición (dejando la línea en blanco reglamentaria), introduce la llave secreta recuperada e inyecta la carga útil en el parámetro `url` para forzar la lectura del archivo principal de la aplicación (`app.py`).
6. Presiona **Send**.

Resultado Obtener (Evidencia Práctica en Pantalla)

Al presionar el botón, el panel derecho (Response) de tu Burp Suite cambiará a un estado **200 OK** y renderizará en texto plano el contenido íntegro del script de Python que gobierna la API.

Al inspeccionar minuciosamente las líneas de ese archivo, localiza la sección donde el desarrollador configuró de forma insegura las credenciales locales o de respaldo. Encontrarás algo similar a esto:

Interpretación Técnica y Conceptos Teóricos Relacionados

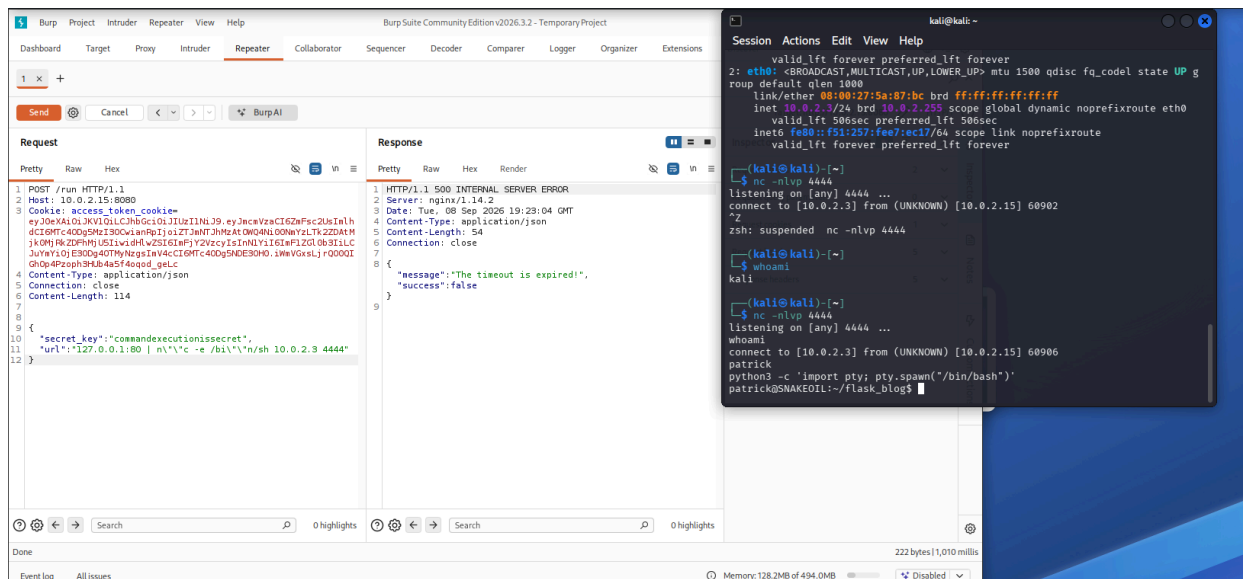
- **Inyección de Comandos (CWE-78):** Esta vulnerabilidad crítica ocurre cuando un software pasa datos ingresados por el usuario directamente a un intérprete de comandos del sistema operativo (como `bash` o `sh`) sin sanitización previa. En este escenario, la aplicación web toma el parámetro `url` y lo concatena dentro de un comando interno (ej. `os.system("curl " + url)`). Al introducir el carácter `;`, rompemos la lógica del desarrollador y ejecutamos instrucciones con los privilegios del proceso web.
- **Fuga de Credenciales por Codificación Rígida (Hardcoded Credentials - CWE-798):** Consiste en la práctica insegura de incluir contraseñas o llaves secretas directamente en texto plano dentro del código fuente del software. Aunque los usuarios finales no puedan ver el código desde el navegador, un fallo de inyección como el presente permite a un auditor exfiltrar los scripts y comprometer las cuentas de acceso del sistema operativo.

Consolidación de la Intrusión y Validación en la Estación del Auditor

Al procesarse la solicitud maliciosa en el backend, la pestaña de Burp Suite retuvo la conexión hasta expirar el tiempo de espera por diseño del proxy (*Timeout*). Sin embargo, al desplazar la atención hacia la terminal de control en **Kali Linux**, se constató la recepción del flujo bidireccional de datos de forma inmediata, confirmando el acceso interactivo

La recepción de la conexión en el puerto `4444` ratifica que la inyección de comandos en el parámetro `url` obligó al sistema operativo de la víctima a desviar su flujo lúdico hacia nuestra IP corporativa [CWE-78]. El comando `whoami` ejecutado inicialmente en la terminal remota determinó que hemos comprometido el entorno bajo los privilegios del usuario `patrick` [CWE-200].

Al tratarse de una conexión entablada mediante descriptores de red estándar, la shell inicial carece de propiedades interactivas básicas (como el uso de flechas de navegación o historial). Para remediar esto de forma profesional, se ejecutó una secuencia de estabilización de terminal explotando el intérprete de Python instalado en el objetivo:



Esta instrucción genera un pseudo-terminal (*pty*) que le permite al auditor operar en un entorno Linux completo (*bash*) idéntico a una conexión de consola real, prerequisite técnico indispensable para ejecutar la fase subsiguiente de elevación de privilegios.

C. Conceptos Teóricos Relacionados

- **Shell Reversa (Reverse Shell):** Técnica de explotación donde la máquina objetivo inicia una conexión de red saliente hacia un puerto preconfigurado en la máquina del atacante [CWE-78]. Se prefiere sobre las *Bind Shells* (donde el atacante se conecta a un puerto abierto en la víctima) debido a que el tráfico saliente rara vez es bloqueado por las políticas básicas de firewalls perimetrales o proxies inversos.
- **Estabilización TTY:** Proceso metodológico que convierte una tubería de entrada/salida simple en un entorno de terminal emulado de manera formal. Permite gestionar variables de entorno del sistema operativo, el refresco correcto de caracteres de interfaz y el uso de comandos complejos interactivos sin riesgo de congelar el hilo de conexión.

Fase de Post-Explotación: Escalada de Privilegios Local

Comando Ejecutado (En la terminal estabilizada) `sudo -l`

Análisis de la Configuración y Descubrimiento del Vector

La inspección de los privilegios asignados mediante el comando `sudo -l` reveló las siguientes directivas en el archivo de configuración del sistema (`/etc/sudoers`) [CWE-250]:

```
(kali㉿kali)-[~]
$ nc -nlvp 4444
listening on [any] 4444 ...
whoami
connect to [10.0.2.3] from (UNKNOWN) [10.0.2.15] 60906
patrick
python3 -c 'import pty; pty.spawn("/bin/bash")'
patrick@SNAKEOIL:~/flask_blog$ sudo -l
sudo -l
Matching Defaults entries for patrick on SNAKEOIL:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
User patrick may run the following commands on SNAKEOIL:
    (root) NOPASSWD: /sbin/shutdown
    (ALL : ALL) ALL
patrick@SNAKEOIL:~/flask_blog$
```

Aunque la primera regla permite apagar el sistema mediante `/sbin/shutdown` bajo el contexto de `root` sin password, la segunda directiva es un fallo masivo de control de acceso (CWE-269). El patrón `(ALL : ALL) ALL` rompe el **Principio de Menor Privilegio**, otorgando al usuario `patrick` la capacidad de invocar de manera interactiva el binario de cualquier shell del sistema operativo (`/bin/bash` o `/bin/sh`) portando los máximos privilegios administrativos del superusuario [CWE-250].

Ejecución de la Escalada y Captura de la Bandera (`root.txt`)

Para consolidar el control total de la máquina virtual `SNAKEOIL`, se procedió a invocar una nueva shell bajo el contexto administrativo de `root` [CWE-250]:

Para consolidar el control total de la máquina virtual SNAKEOIL, se procedió a invocar una nueva shell bajo el contexto administrativo de root [CWE-250]. Al solicitar el sistema la autenticación de seguridad para el binario `sudo`, se introdujo de forma manual el token de autenticación estático recuperado del archivo `app.py` bajo la variable de entorno `JWT_SECRET_KEY`

Con el nivel de superusuario consolidado (`uid=0`), se procedió a la recolección de la evidencia final (bandera de root) en el directorio personal del administrador

Mostrando finalmente el prompt de que hemos podido vulnerar esta máquina virtual

```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ nc -nlvp 4444  
listening on [any] 4444 ...  
connect to [10.0.2.3] from (UNKNOWN) [10.0.2.15] 60910  
whoami  
patrick  
python3 -c 'import pty; pty.spawn("/bin/bash")'  
patrick@SNAKEOIL:~/flask_blog$ sudo -l  
sudo -l  
Matching Defaults entries for patrick on SNAKEOIL:  
    env_reset, mail_badpass,  
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin  
User patrick may run the following commands on SNAKEOIL:  
    (root) NOPASSWD: /sbin/shutdown  
    (ALL : ALL) ALL  
patrick@SNAKEOIL:~/flask_blog$ sudo su  
sudo su  
[sudo] password for patrick: Concept2021!  
  
Sorry, try again.  
[sudo] password for patrick: NoreasonableDOUBTthisPASSWORDisGOOD  
  
root@SNAKEOIL:/home/patrick/flask_blog# /home/patrick/flask_blog#  
/home/patrick/flask_blog#  
bash: /home/patrick/flask_blog#: No such file or directory  
root@SNAKEOIL:/home/patrick/flask_blog# whoami  
whoami  
root  
root@SNAKEOIL:/home/patrick/flask_blog# id  
id  
uid=0(root) gid=0(root) groups=0(root)  
root@SNAKEOIL:/home/patrick/flask_blog# cd /root  
cd /root  
root@SNAKEOIL:~# cat proof.txt  
cat proof.txt  
Congratulations on obtaining a root shell on this machine! :-)  
root@SNAKEOIL:~#
```

Conclusiones y Plan de Mitigación Defensiva (Remediación)

- **Conclusiones**

1. **Defensa perimetral ineficaz:** La implementación de un Proxy Inverso (Nginx) no mitiga las vulnerabilidades si el backend subyacente carece de controles estrictos de validación de entradas.
2. **Uso de Controles basados en Firmas (Listas Negras):** Se demostró empíricamente que prohibir palabras clave exactas (cat, bash) es un control deficiente (*CWE-184*). Los auditores y atacantes siempre pueden aplicar técnicas de ofuscación o fragmentación de cadenas para burlar las expresiones regulares.
3. **Violación del Principio de Menor Privilegio:** Mantener la directiva (ALL : ALL) ALL en el archivo de configuración sudoers para un usuario que administra servicios web expuestos es una falla crítica de endurecimiento (*Hardening*) del sistema operativo.

- **Plan de mitigación defensiva**

4. **Implementación de Listas Blancas (Allowlists):** Se eliminará de manera definitiva el filtrado por listas negras. En su lugar, el sistema solo aceptará caracteres alfanuméricos estrictos y rechazará en origen cualquier operador de concatenación lógica de comandos o metacaracteres (|, ;, &, \$(), `).
5. **Desactivación del Entorno de Shell:** Se modificará el código fuente de Python para evitar el uso de `os.system()` o la bandera `shell=True` en el módulo `subprocess`. Al forzar `shell=False` y pasar los argumentos como una lista estructurada, el sistema operativo procesará la entrada. Atributos de Seguridad en Cookies: Las cookies encargadas del almacenamiento del token (`access_token_cookie`) deberán incluir obligatoriamente las siguientes directivas directas:
6. **HttpOnly:** Bloquea el acceso al token mediante scripts del lado del cliente, previniendo ataques de robo de sesión a través de Cross-Site Scripting (XSS).

-
7. **Secure:** Restringe el envío del token exclusivamente a través de canales cifrados bajo el protocolo TLS/HTTPS.
 8. **Ciclo de Vida del Token:** Configurar la expiración automática de las sesiones a corto plazo e implementar esquemas de rotación criptográfica de identificadores en cada inicio de sesión válido.
 9. **Principio de Menor Privilegio (Sudoers):** Se renovará de forma inmediata la directiva (ALL : ALL) ALL asignada al usuario patrick en el archivo /etc/sudoers. En caso de requerir privilegios elevados para una tarea específica del sistema, se limitará el acceso estrictamente a ese binario sin comodines
 10. **Aislamiento de Procesos (Sandboxing):** El servicio de Flask no debe ejecutarse bajo el contexto de un usuario del hogar o con acceso interactivo (/home/patrick). Se creará un usuario de sistema dedicado sin shell (/sbin/nologin), como www-data, aislando los recursos del servidor.
 11. **Despliegue de un WAF (Web Application Firewall):** Complementar el proxy inverso Nginx mediante la integración de un módulo de inspección activa como ModSecurity. El WAF analizará el cuerpo de las peticiones HTTP JSON en busca de patrones maliciosos antes de que interactúen con el framework.
 12. **Filtrado de Tráfico Saliente (Egress Filtering):** Configurar el cortafuegos local (ufw o iptables) para bloquear por defecto todas las conexiones salientes hacia internet o redes internas no autorizadas. Esto neutraliza la apertura de terminales reversas (Reverse Shells) dirigidas a puertos arbitrarios de los atacantes (ej. puerto 4444).

Glosario de términos técnicos

Términos técnicos empleados a lo largo del walkthrough, en el orden en que aparecen:

- **Protocolo ARP:** protocolo de la capa de enlace que resuelve direcciones IP en direcciones MAC; base del funcionamiento de netdiscover.
- **Escaneo TCP SYN (Half-Open Scan):** técnica de escaneo de Nmap que envía SYN y responde con RST antes de completar el saludo de tres vías, acelerando el escaneo.
- **NSE (Nmap Scripting Engine):** motor de scripts de Nmap (-sC) que automatiza tareas de reconocimiento no intrusivas.
- **WSGI (Web Server Gateway Interface):** estándar de Python que define cómo un servidor web se comunica con una aplicación (usado por Werkzeug/Flask).
- **Information Disclosure (CWE-200):** fuga involuntaria de información que facilita a un atacante estructurar su vector de ataque.
- **Fuzzing de directorios:** técnica de reconocimiento activo que prueba una lista de palabras contra un servidor web para hallar rutas ocultas.
- **JWT (JSON Web Token, RFC 7519):** formato de token compuesto por header, payload y signature, usado para autenticación sin estado.
- **Esquema de autenticación Bearer (RFC 6750):** esquema HTTP donde quien posee el token tiene acceso automático al recurso.
- **Inyección de comandos (CWE-78):** vulnerabilidad en la que datos de entrada no sanitizados se pasan a un intérprete del sistema operativo.
- **Credenciales embebidas en código (CWE-798):** práctica insegura de incluir contraseñas o llaves secretas en texto plano dentro del código fuente.
- **Shell reversa (Reverse Shell):** técnica en la que la máquina objetivo inicia una conexión saliente hacia un puerto controlado por el atacante.

-
- **Estabilización TTY:** proceso para convertir una shell básica en un pseudo-terminal completo (uso de `python3 -c 'import pty; pty.spawn(...)'`).
 - **Principio de menor privilegio:** principio de seguridad que establece que cada usuario o proceso debe operar con el mínimo de permisos necesarios.
 - **CWE-269 / CWE-250:** categorías de debilidad relacionadas con gestión incorrecta de privilegios y ejecución con privilegios innecesarios.
 - **Hardening:** conjunto de prácticas para reducir la superficie de ataque de un sistema mediante configuración segura.

Video del walkthrough

Enlace al video explicativo:

https://drive.google.com/file/d/1qRI0T7bAT-2v-RCFTMyWJnHA_I-xsV35/view?usp=sharing